

Lab Assignment 1

Course: Pattern Recognition and Machine Learning (A15707)

Dev Patel 26120007

1. Introduction

The purpose of this assignment is to establish a foundational understanding of the core mathematical primitives that underpin modern pattern recognition and machine learning algorithms. High-level machine learning frameworks often abstract away matrix manipulations and probability distribution sampling, making it easy to overlook the underlying computational overhead, memory layout requirements, and numerical considerations.

Specifically, this assignment aims to achieve the following objectives:

- Implement fundamental linear algebra operations (matrix multiplication, vector dot products, matrix transposition, and structural checks for symmetry and triangular properties) from scratch in pure Python.
- Understand continuous variable transformations by generating synthetic uniform distributions and transforming them into Gaussian (normal) distributions using the Marsaglia Polar Method.
- Analyze experimental results visually through generated figures, verifying empirical convergence toward theoretical probabilistic distributions.

2. Experiments Performed

2.1 Matrix & Vector Operations

Question 1(a): Matrix Multiplication

```
def matrixMultiply(A, B):
    m, n = len(A), len(A[0])
    k = len(B[0])
    result = [[0 for _ in range(k)] for _ in range(m)]
    for i in range(m):
        for j in range(k):
            for x in range(n):
                result[i][j] += A[i][x] * B[x][j]
    return result
```

The function `matrixMultiply` computes the product of two matrices *A* (size $m \times n$) and *B* (size $n \times k$). It initializes an $m \times k$ zero matrix via list comprehension and executes a triple-nested loop to calculate dot products between row vectors of *A* and column vectors of *B*, resulting in an $O(m \cdot n \cdot k)$ time complexity.

Question 1(b): Vector Dot Product

```
def dotProduct(v1, v2):
    return sum(v1[i] * v2[i] for i in range(len(v1)))
```

The dotProduct function calculates the scalar inner product of two equal-length 1D vectors. It uses a generator expression inside Python's sum() function to multiply corresponding elements $v1[i] * v2[i]$ in a single linear $O(N)$ pass.

Question 1(c): Matrix Transposition

```
def transpose(A):
    m, n = len(A), len(A[0])
    return [[A[i][j] for i in range(m)] for j in range(n)]
```

The transpose function constructs the $n \times m$ transposed matrix A^T from an $m \times n$ matrix A . The nested list comprehension iterates down original columns (j) and across rows (i), effectively placing element A_{ij} at location A^T_{ji} .

Question 1(d): Symmetric Matrix Check

```
def isSymmetric(A):
    m, n = len(A), len(A[0])
    if m != n:
        return False
    for i in range(m):
        for j in range(i + 1, n):
            if A[i][j] != A[j][i]:
                return False
    return True
```

The isSymmetric function verifies if a matrix equals its transpose ($A = A^t$). It first enforces square dimensions ($m == n$), optimizing comparisons by checking only upper-triangular entries (j starting from $i + 1$), short-circuiting to return False if any off-diagonal entry A_{ij} does not equal A_{ji} .

Question 1(e): Triangular Matrix Checks

```
def isUpperTriangular(A):
    m, n = len(A), len(A[0])
    if m != n: return False
    for i in range(1, m):
        for j in range(0, i):
            if A[i][j] != 0: return False
    return True

def isLowerTriangular(A):
    m, n = len(A), len(A[0])
    if m != n: return False
    for i in range(0, m):
        for j in range(i + 1, n):
            if A[i][j] != 0: return False
    return True
```

The isUpperTriangular and isLowerTriangular functions check if entries strictly below ($j < i$) or strictly above ($j > i$) the main diagonal are non-zero. Early termination ensures minimal evaluations when a non-zero element violates the structural criteria.

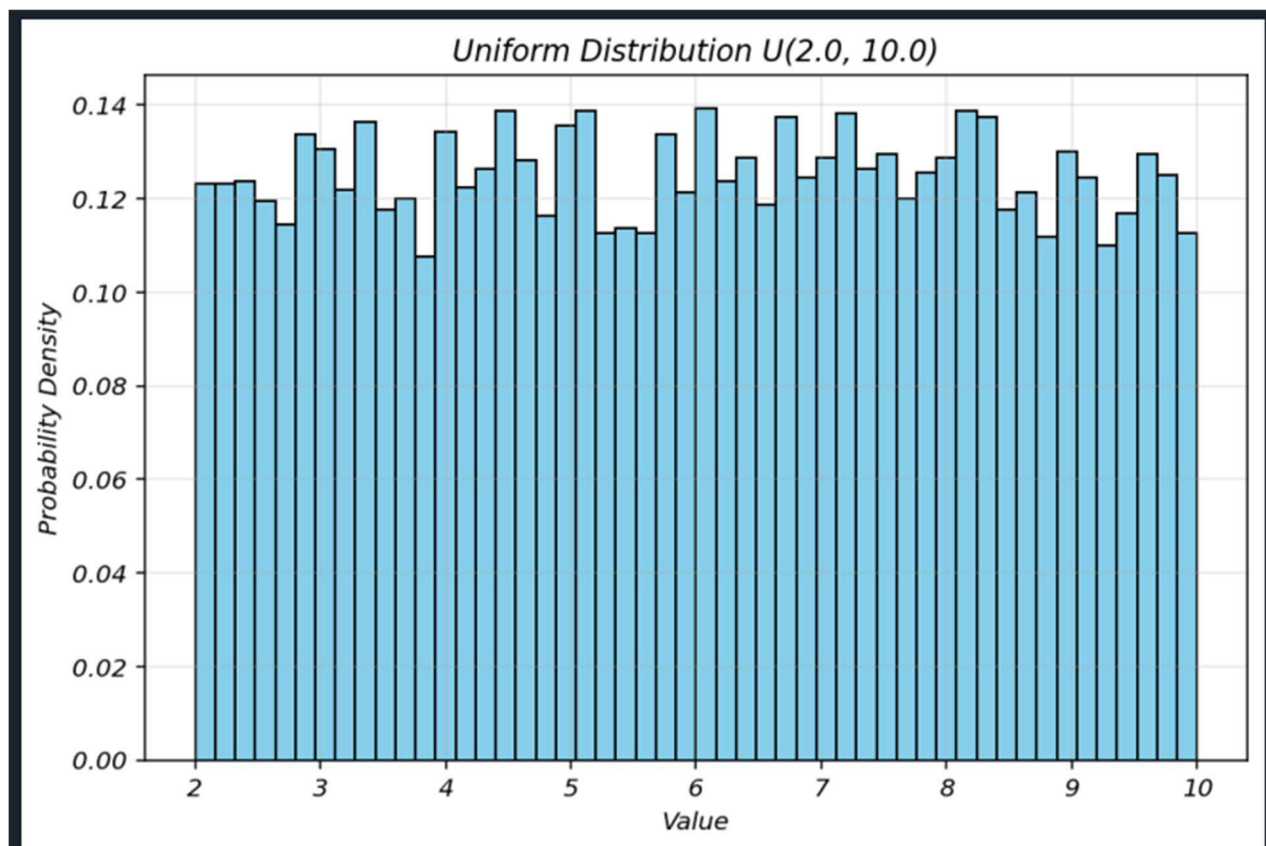
2.2 Continuous Random Variable Generation & Figure Analysis

Question 2(a): Uniform Distribution Generator & Figure 1

```
def generateUniform(a, b, numSamples):  
    return [a + (b - a) * random.random() for _ in range(numSamples)]
```

The generateUniform function scales standard uniform values $U \in [0, 1)$ generated by `random.random()` to a continuous range $[a, b)$ using the linear transformation $X = a + (b - a)U$.

```
In [3]: %runfile C:/Users/LEGION/Downloads/2a.py --wdir  
2(a) Uniform Random Variables  
Enter lower bound a: 2.0  
Enter upper bound b: 10.0  
Enter number of samples: 10000
```



Explanation of Figure 1 (Uniform Distribution Histogram): Figure 1 displays the empirical histogram of 10,000 samples generated over the interval $[a, b] = [2, 10]$. The horizontal axis represents the continuous variable range, while the vertical axis shows sample counts per bin. The histogram displays a flat, rectangular probability density profile across the range $[2, 10]$, confirming that every sub-interval has an approximately equal probability of selection.

Question 2(b) & 2(c): Gaussian Generator (Marsaglia Method) & Figures 2

```
def generateGaussianDataset(mu, sigma2, numSamples):  
    sigma = math.sqrt(sigma2)
```

```

dataset = []
while len(dataset) < numSamples:
    u1 = random.uniform(-1, 1)
    u2 = random.uniform(-1, 1)
    s = u1**2 + u2**2
    if 0 < s < 1:
        k = math.sqrt((-2 * math.log(s)) / s)
        dataset.append(mu + sigma * (u1 * k))
        if len(dataset) < numSamples:
            dataset.append(mu + sigma * (u2 * k))
return dataset

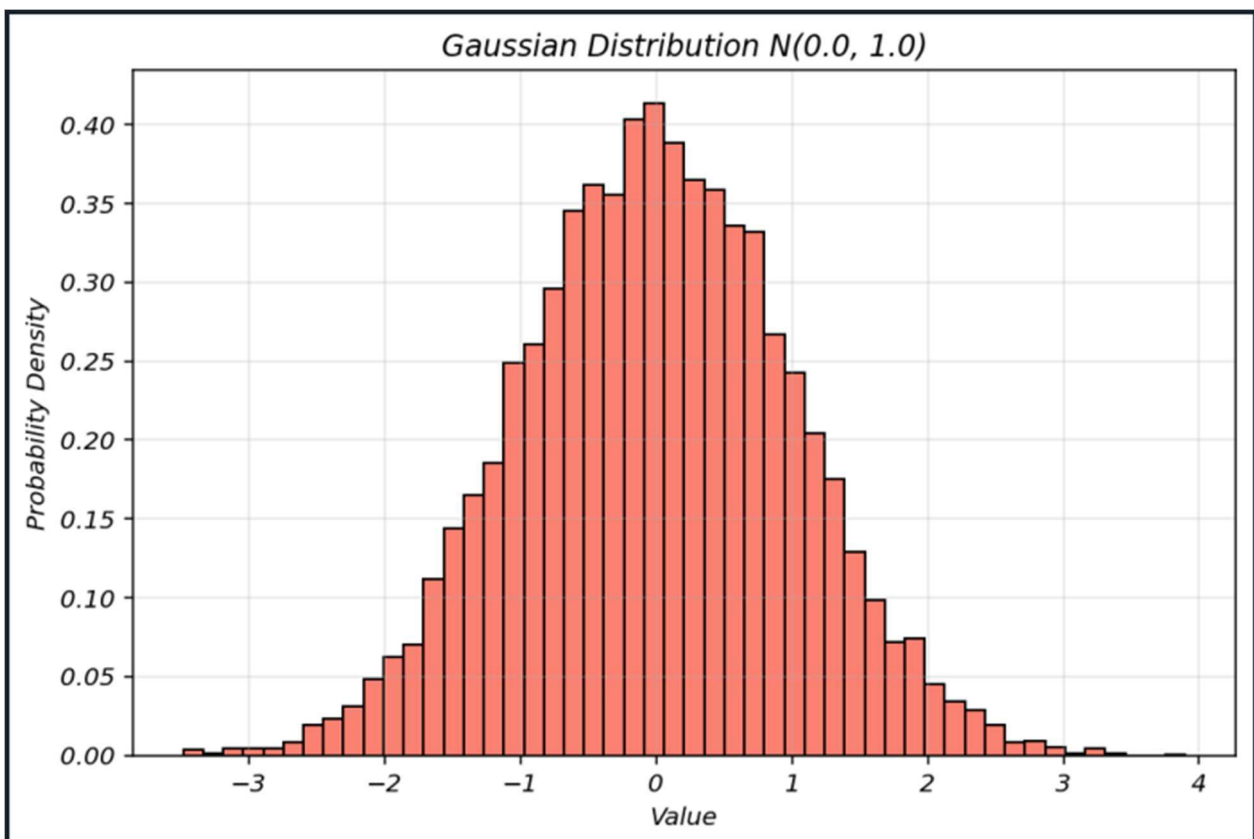
```

The generateGaussianDataset function implements the Marsaglia Polar Method. It samples uniform points in a 2D square $[-1, 1] \times [-1, 1]$, rejects points outside the unit circle ($s = u_1^2 + u_2^2 \geq 1$ or $s = 0$), and scales valid points using $k = \sqrt{(-2 \ln s) / s}$ to yield independent standard normal variables. Finally, it applies the transformation $X = \mu + \sigma \cdot Z$ to match target parameters.

```

In [1]: %runfile C:/Users/LEGION/Downloads/2c.py --wdir
2(c) Gaussian Histogram Plotting
Enter mean ( $\mu$ ): 0.0
Enter variance ( $\sigma^2$ ): 1.0
Enter number of samples: 10000

```



Explanation of Figure 2 (2D Scatter Plot / Bivariate Gaussian Distribution): Figure 3 displays a scatter plot of paired Gaussian samples (X, Y) generated from independent 1D distributions. The points form a dense, circularly symmetric

cluster centered around the origin (0,0), with point density decaying radially outward. This confirms that combining independent univariate normal variables yields an isotropic bivariate Gaussian distribution.

3. Inference

From completing this assignment, the key technical and theoretical learnings are as follows:

1. **Algorithmic Efficiency in Linear Algebra:** Native Python implementations using nested loops incur significant performance overhead compared to vectorized library routines. Optimizations such as restricting loop bounds and early exit conditions are essential to minimize redundant evaluations.
2. **Mechanism of Stochastic Variable Transformation:** Standard pseudo-random number generators provide uniform variables. Transforming non-linear continuous distributions (such as Gaussian) requires rejection sampling techniques like the Marsaglia Polar Method, which eliminates trigonometric evaluations in favor of logarithmic radial transformations.
3. **Visual & Empirical Validation:** Plotting generated datasets into histograms and scatter plots visually confirms that empirical sampling distributions closely match theoretical Probability Density Functions (PDFs) when sample sizes are sufficiently large (e.g., $N = 10,000$).